UNIT 2

PROGRAMMING IN C WITH DATA STRUCTURE

Index

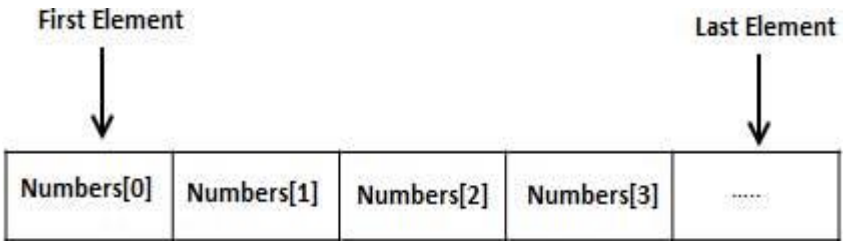
Topic	Page no.
Array and its types	2-7
Passing arrays to a function in C	7-8
Strings	8
Passing string to a function in C	9
Strings Functions	9-15
Pointers	15-16
Pointer Arithmetic	16-19
Array of pointer	19-20
Function returning pointers	20-21
Passing Pointer to a Function	21-22
Dynamic Memory Allocation	22-25
Structures and unions	26-28
File handling	29-34

Array:

Arrays a kind of data structure that can store a fixed-size sequential collection of elements of the same type. An array is used to store a collection of data.

Instead of declaring individual variables, such as number0, number1, ..., and number99, you declare one array variable such as numbers and use numbers [0], numbers [1], and ..., numbers [99] to represent individual variables. A specific element in an array is accessed by an index.

All arrays consist of contiguous memory locations. The lowest address corresponds to the first element and the highest address to the last element.

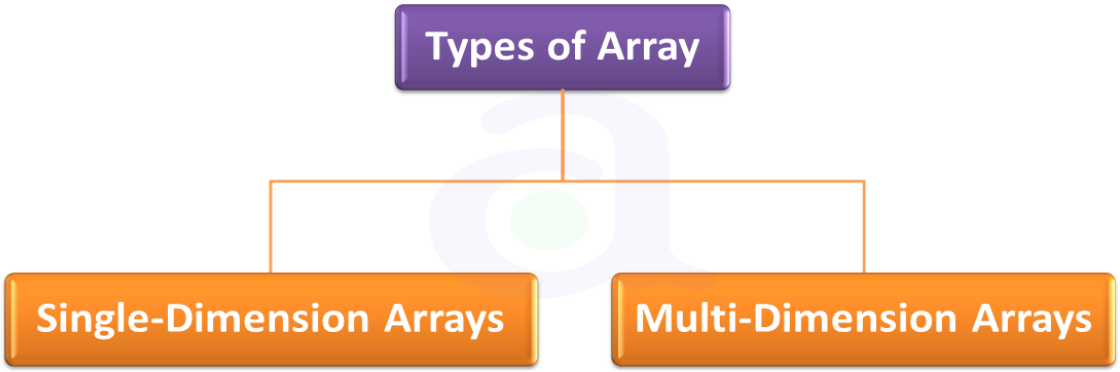


Declaring Arrays:

To declare an array in C, a programmer specifies the type of the elements and the number of elements required by an array as follows –

```
type arrayName [ arraySize];
```

Types of Arrays:



Single-Dimension Arrays:

A one-dimensional array is a structured collection of components (often called array elements) that can be accessed individually by specifying the position of a component with a single index value.

Syntax: 1.data-type arr_name[array_size];

2. data_type array_name [array_size]=(list of elements of an array);

Example: 1. int marks[5];

2. int marks[5]={90,80,50,80,70};

Program 1:

```
#include<stdio.h>
# include<conio.h>
int main(){
int i=0;
int marks[5]={20,30,40,50,60};
for(i=0;i<5;i++){
printf("%d \n",marks[i]);
}
return 0;
}
```

Output:

```
20
30
40
50
60
```

Program 2:

```
#include<stdio.h>
 # include<conio.h>
int main()
{
   int arr[5], i;

   for(i = 0; i < 5; i++)
   {
      printf("Enter a[%d]: ", i);
      scanf("%d", &arr[i]);
   }

   printf("\nPrinting elements of the array: \n\n");

   for(i = 0; i < 5; i++)
   {
      printf("%d ", arr[i]);
   }

   // signal to operating system program ran fine
   return 0;
}
```

Output:

```
Enter a[0]: 11
Enter a[1]: 22
Enter a[2]: 34
Enter a[3]: 4
Enter a[4]: 34

Printing elements of the array:

11 22 34 4 34
```

Two-Dimensional Array:

The two-dimensional array can be defined as an array of arrays. The 2D array is organized as matrices which can be represented as the collection of rows and columns.

Syntax: data_type array_name[rows][columns];

Example: int table[4][3];
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};

Program 1:

```
#include<stdio.h>
#include<conio.h>
int main(){
int I,j;
int arr[4][3]={{1,2,3},{2,3,4},{3,4,5},{4,5,6}};
for(i=0;i<4;i++){
 for(j=0;j<3;j++){
   printf("arr[%d] [%d] = %d \n",i,j,arr[i][j]);
 }//end of j
}//end of i
return 0;
}
```

Output:

```
arr[0][0] = 1
arr[0][1] = 2
arr[0][2] = 3
arr[1][0] = 2
arr[1][1] = 3
arr[1][2] = 4
arr[2][0] = 3
arr[2][1] = 4
arr[2][2] = 5
arr[3][0] = 4
arr[3][1] = 5
arr[3][2] = 6
```

Program 2:

```
 #include <stdio.h>
#include <conio.h>
void main ()
{
    int arr[3][3],i,j;
    for (i=0;i<3;i++)
    {
        for (j=0;j<3;j++)
        {
            printf("Enter a[%d][%d]: ",i,j);
            scanf("%d",&arr[i][j]);
        }
    }
    printf("\n printing the elements ....\n");
    for(i=0;i<3;i++)
    {
        printf("\n");
        for (j=0;j<3;j++)
        {
            printf("%d\t",arr[i][j]);
        }
    }
}
```

Output:

```
Enter a[0][0]: 56
Enter a[0][1]: 10
Enter a[0][2]: 30
Enter a[1][0]: 34
Enter a[1][1]: 21
Enter a[1][2]: 34

Enter a[2][0]: 45
Enter a[2][1]: 56
Enter a[2][2]: 78

 printing the elements ....

56      10      30
34      21      34
45      56      78
```

Passing arrays to a function in C

Passing array elements to a function is similar to passing variables to a function.

Program:

```c
#include <stdio.h>
void display(int age1, int age2) {
  printf("%d\n", age1);
  printf("%d\n", age2);
}

int main() {
  int ageArray[] = {2, 8, 4, 12};

  // pass second and third elements to display()
  display(ageArray[1], ageArray[2]);
  return 0;
}
```

Output:

```
8
4
```

Strings

Strings are actually one-dimensional array of characters terminated by a null character.

Program:

```
#include<stdio.h>
#include<conio.h>
#include <string.h>
int main(){
  char ch1[11]={'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0'};
   char ch2[11]="javatpoint";

   printf("Char Array Value is: %s\n", ch1);
   printf("String Literal Value is: %s\n", ch2);
 return 0;
}
```

Output:

```
Char Array Value is: javatpoint
String Literal Value is: javatpoint
```

Passing string to a Function

Strings can be passed to a function in a similar way as arrays.

Program:

```
#include <stdio.h>
#include <conio.h>
void displayString(char str[]);

int main()
{
    char str[50];
    printf("Enter string: ");
    fgets(str, sizeof(str), stdin);
    displayString(str);     // Passing string
to a function.
    return 0;
}
void displayString(char str[])
{
    printf("String Output: ");
    puts(str);
}
```

String Functions:

1. String Length: strlen() function:

The strlen() function returns the length of the given string.

Program:

```
#include<stdio.h>

#include<conio.h>
```

```
#include <string.h>
int main(){
char ch[20]={'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0'};
  printf("Length of string is: %d",strlen(ch));
 return 0;
}
```

Output:

```
Length of string is: 10
```

2. Copy String: strcpy() function:

The strcpy(destination, source) function copies the source string in destination.

Program:

```
#include<stdio.h>
#include<conio.h>
#include <string.h>
int main(){
 char ch1[20]={'j', 'a', 'v', 'a', 't', 'p', 'o', 'i', 'n', 't', '\0'};
  char ch2[20];
  strcpy(ch2,ch1);
  printf("Value of second string is: %s",ch2);
 return 0;
}
```

Output:

```
Value of second string is: javatpoint
```

3. String Concatenation: strcat() function:

The strcat(first_string, second_string) function concatenates two strings and result is returned to first_string.

Program:

```
#include<stdio.h>
#include<conio.h>
#include <string.h>
int main(){
  char ch1[10]={'h', 'e', 'l', 'l', 'o'};
   char ch2[10]={'c'};
   strcat(ch,ch2);
   printf("Value of first string is: %s",ch1);
 return 0;
}
```

Output:

```
Value of first string is: helloc
```

4. Compare String: strcmp() function:

The strcmp(first_string, second_string) function compares two string and returns 0 if both strings are equal.

Program:

```
#include<stdio.h>
#include<conio.h>
#include <string.h>
int main(){
  char str1[20],str2[20];
  printf("Enter 1st string: ");
  gets(str1);//reads string from console
  printf("Enter 2nd string: ");
  gets(str2);
  if(strcmp(str1,str2)==0)
     printf("Strings are equal");
  else
     printf("Strings are not equal");
 return 0;
}
```

Output:

```
Enter 1st string: hello
Enter 2nd string: hello
Strings are equal
```

5. Reverse String: strrev() function:

The strrev(string) function returns reverse of the given string. Let's see a simple example of strrev() function.

Program:

```
#include<stdio.h>
#include<conio.h>
#include <string.h>
int main(){
  char str[20];
  printf("Enter string: ");
  gets(str);//reads string from console
  printf("String is: %s",str);
  printf("\nReverse String is: %s",strrev(str));
 return 0;
}
```

Output:

```
Enter string: javatpoint
String is: javatpoint
Reverse String is: tnioptavaj
```

6. String Lowercase: strlwr() function:

The strlwr(string) function returns string characters in lowercase. Let's see a simple example of strlwr() function.

Program:

```
#include<stdio.h>
#include<conio.h>
#include <string.h>
int main(){
```

```
  char str[20];
  printf("Enter string: ");
  gets(str);//reads string from console
  printf("String is: %s",str);
  printf("\nLower String is: %s",strlwr(str));
 return 0;
}
```

Output:

```
Enter string: JAVATpoint
String is: JAVATpoint
Lower String is: javatpoint
```

7. String Uppercase: strupr() function:

The strupr(string) function returns string characters in uppercase. Let's see a simple example of strupr() function.

Program:

```
#include<stdio.h>
#include<conio.h>
#include <string.h>
int main(){
  char str[20];
  printf("Enter string: ");
  gets(str);//reads string from console
  printf("String is: %s",str);
  printf("\nUpper String is: %s",strupr(str));
 return 0;    }
```

Output:

```
Enter string: javatpoint
String is: javatpoint
Upper String is: JAVATPOINT
```

Pointers

The pointer in C language is a variable which stores the address of another variable. This variable can be of type int, char, array, function, or any other pointer.

Syntax:

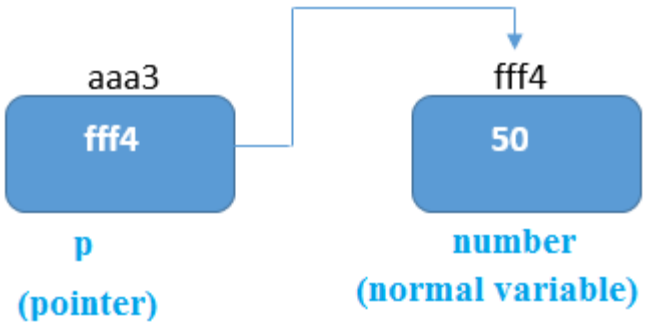
int *a;//pointer to int

char *c;//pointer to char

example:

int number = 10;

int* p = &number;



Program:

#include<stdio.h>

#include<conio.h>

int main(){

```
int number=50;
int *p;
p=&number;//stores the address of number variable
printf("Address of p variable is %x \n",p);
printf("Value of p variable is %d \n",*p);
return 0;
}
```

Pointer Arithmetic

We can perform arithmetic operations on the pointers like addition, subtraction, etc. However, as we know that pointer contains the address, the result of an arithmetic operation performed on the pointer will also be a pointer if the other operand is of type integer.

Incrementing Pointer

If we increment a pointer by 1, the pointer will start pointing to the immediate next location.

Syntax:

```
new_address= current_address + i * size_of(data type)
```

Program:

```
#include<stdio.h>
int main(){
int number=50;
int *p;//pointer to int
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p+1;
printf("After increment: Address of p variable is %u \n",p);
return 0;
}
```

Output:

```
Address of p variable is 3214864300
After increment: Address of p variable is 3214864302
```

Decrementing Pointer

If we increment a pointer by 1, the pointer will start pointing to the immediate next location.

Syntax:

new_address= current_address - i * size_of(data type)

Program:

```
#include <stdio.h>
void main(){
int number=50;
int *p;//pointer to int
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p-1;
printf("After decrement: Address of p variable is %u \n",p);
}
```

Output:

```
Address of p variable is 3214864300
After increment: Address of p variable is 3214864208
```

Pointer Addition

We can add a value to the pointer variable. The formula of adding value to pointer is given below:

Syntax:

new_address= current_address + (number * size_of(data type))

Program:

```
#include<stdio.h>
int main(){
int number=50;
int *p;//pointer to int
p=&number;//stores the address of number variable
printf("Address of p variable is %u \n",p);
p=p+3;   //adding 3 to pointer variable
printf("After adding 3: Address of p variable is %u \n",p);
return 0;
}
```

Output:

```
Address of p variable is 3214864300
After increment: Address of p variable is 3214864306
```

Pointer Subtraction

we can subtract a value from the pointer variable. Subtracting any number from a pointer will give an address.

Syntax:

new_address= current_address - (number * size_of(data type))

Program:

```
#include<stdio.h>
int main(){
int number=50;
int *p;//pointer to int
p=&number;
printf("Address of p variable is %u \n",p);
p=p-3;
```

```
    printf("After subtracting 3: Address of p variable is %u \n",p);
        return 0;
    }
```

Output:

```
Address of p variable is 3214864300
After increment: Address of p variable is 3214864204
```

Array of Pointers

Syntax:

```
datatype *array_name[size];
```

Program:

```
#include<stdio.h>
#define SIZE 10

int main()
{
    int *arrop[3];
    int a = 10, b = 20, c = 50, i;

    arrop[0] = &a;
    arrop[1] = &b;
    arrop[2] = &c;

    for(i = 0; i < 3; i++)
    {
        printf("Address = %d\t Value = %d\n", arrop[i], *arrop[i]);
    }
```

```
    return 0;
}
```

Output:

```
Address = 387130656      Value = 10
Address = 387130658     Value = 20
Address = 387130660      Value = 50
```

Function returning pointers

function can return data of types int, float, char etc. Similarly, a function can return a pointer to data. The syntax of a function returning a pointer is as follows.

Syntax:

```
type *function_name(type1, type2, ...);
```

Program:

```
#include<stdio.h>
int *return_pointer(int *, int);

int main()
{
    int i, *ptr;
    int arr[] = {11, 22, 33, 44, 55};
    i = 4;

    printf("Address of arr = %u\n", arr);

    ptr = return_pointer(arr, i);

    printf("\nAfter incrementing arr by 4 \n\n");

    printf("Address of ptr = %u\n\n" , ptr);
    printf("Value at %u is %d\n", ptr, *ptr);
```

```
    return 0;
}

int *return_pointer(int *p, int n)
{
    p = p + n;
    return p;
}
```

Output:

```
Address of arr = 2686736

After incrementing arr by 4
```

Passing Pointer to a Function

we are passing a pointer to a function. When we pass a pointer as an argument instead of a variable then the address of the variable is passed instead of the value. So any change made by the function using the pointer is permanently made at the address of passed variable. This technique is known as call by reference in C.

Program:

```c
#include <stdio.h>
void swap(int *n1, int *n2);

int main()
{
    int num1 = 5, num2 = 10;

    // address of num1 and num2 is passed
    swap( &num1, &num2);

    printf("num1 = %d\n", num1);
    printf("num2 = %d", num2);
    return 0;
}
```

```
void swap(int* n1, int* n2)
{
    int temp;
    temp = *n1;
    *n1 = *n2;
    *n2 = temp;
}
```

Output:

```
num1 = 10
num2 = 5
```

Dynamic Memory Allocation

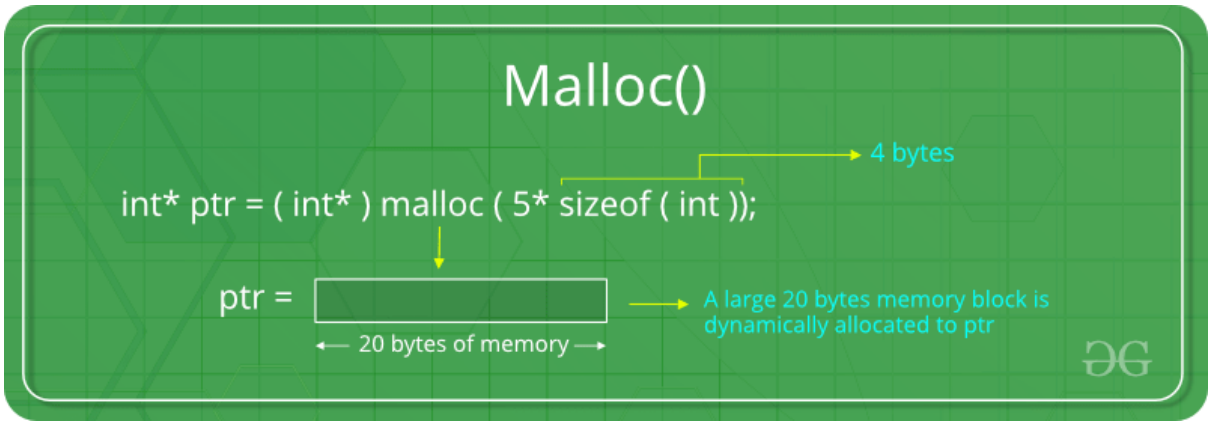
Dynamic Memory Allocation can be defined as a procedure in which the size of a data structure (like Array) is changed during the runtime.

There are 3 library functions provided by C defined under **<stdlib.h>** header file to facilitate dynamic memory allocation in C programming. They are:

1. malloc()
2. calloc()
3. free()

malloc() method:

The **"malloc"** or **"memory allocation"** method in C is used to dynamically allocate a single large block of memory with the specified size. It returns a pointer of type void which can be cast into a pointer of any form.



Syntax:

```
ptr = (cast-type*) malloc(byte-size)
```

Program:

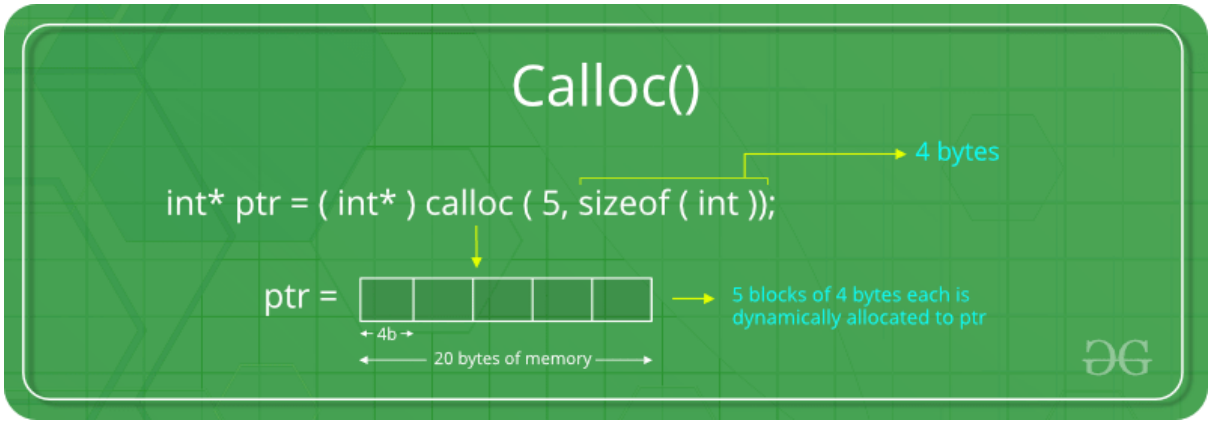
```c
#include <stdio.h>
#include <stdlib.h>

int main()
{

    // This pointer will hold the
    // base address of the block created
    int* ptr;
    int n, i;

    // Get the number of elements for the array
    printf("Enter number of elements:");
    scanf("%d",&n);
    printf("Entered number of elements: %d\n", n);

    // Dynamically allocate memory using malloc()
    ptr = (int*)malloc(n * sizeof(int));

    // Check if the memory has been successfully
    // allocated by malloc or not
    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        // Memory has been successfully allocated
        printf("Memory successfully allocated using malloc.\n");

        // Get the elements of the array
        for (i = 0; i < n; ++i) {
            ptr[i] = i + 1;
        }

        // Print the elements of the array
        printf("The elements of the array are: ");
        for (i = 0; i < n; ++i) {
            printf("%d, ", ptr[i]);
        }
    }
  return 0;
}
```

Output:

```
Enter number of elements: 5
Memory successfully allocated using malloc.
The elements of the array are: 1, 2, 3, 4, 5,
```

calloc() method:

The calloc() function allocates multiple block of requested memory. It initially initialize all bytes to zero.



Syntax:

ptr=(cast-type*)calloc(number, byte-size)

Program:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{

    // This pointer will hold the
    // base address of the block created
    int* ptr;
    int n, i;

    // Get the number of elements for the array
    n = 5;
    printf("Enter number of elements: %d\n", n);
```

```
    // Dynamically allocate memory using calloc()
    ptr = (int*)calloc(n, sizeof(int));

    // Check if the memory has been successfully
    // allocated by calloc or not
    if (ptr == NULL) {
        printf("Memory not allocated.\n");
        exit(0);
    }
    else {

        // Memory has been successfully allocated
        printf("Memory successfully allocated using calloc.\n");

        // Get the elements of the array
        for (i = 0; i < n; ++i) {
            ptr[i] = i + 1;
        }

        // Print the elements of the array
        printf("The elements of the array are: ");
        for (i = 0; i < n; ++i) {
            printf("%d, ", ptr[i]);
        }
    }

    return 0;
}
```

Output:

```
Enter number of elements: 5
Memory successfully allocated using calloc.
The elements of the array are: 1, 2, 3, 4, 5,
```

free() method:

The memory occupied by malloc() or calloc() functions must be released by calling free() function. Otherwise, it will consume memory until program exit.

Syntax:

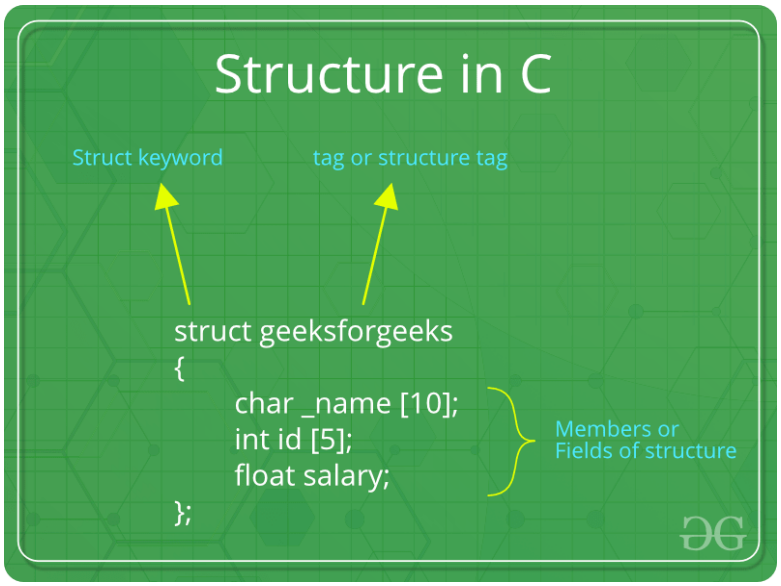
free(ptr)

Structures and unions

Structure: Structure in c is a user-defined data type that enables us to store the collection of different data types.

Syntax:

```
struct struct_name{
   type element1;
   type element2;
   .
   .
} variable1, variable2, ...;
```



Program:

```
#include <stdio.h>
#include <string.h>

// create struct with person1 variable
struct Person {
  char name[50];
  int citNo;
  float salary;
} person1;
```

```c
int main() {

  // assign value to name of person1
  strcpy(person1.name, "George Orwell");

  // assign values to other person1 variables
  person1.citNo = 1984;
  person1. salary = 2500;

  // print struct variables
  printf("Name: %s\n", person1.name);
  printf("Citizenship No.: %d\n", person1.citNo);
  printf("Salary: %.2f", person1.salary);

  return 0;
}
```

Output:

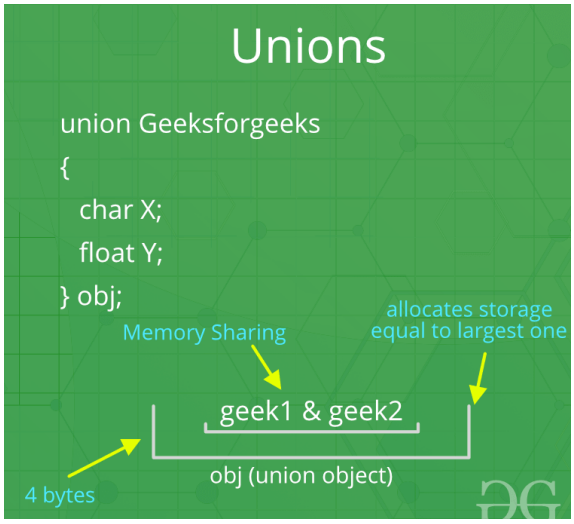
```
Name: George Orwell
Citizenship No.: 1984
Salary: 2500.00
```

Unions:

Union can be defined as a user-defined data type which is a collection of different variables of different data types in the same memory location.

Syntax:

```
union [union name]
    {
        member definition;
        member definition;
        ...
        member definition;
    };
```



Program:

```
#include <stdio.h>
#include <string.h>

union Data {
   int i;
   float f;
   char str[20];
};

int main( ) {

   union Data data;

   data.i = 10;
   printf( "data.i : %d\n", data.i);

   data.f = 220.5;
   printf( "data.f : %f\n", data.f);

   strcpy( data.str, "C Programming");
   printf( "data.str : %s\n", data.str);

   return 0;
}
```

Output:

```
data.i : 10
data.f : 220.500000
data.str : C Programming
```

File Handling

File handling in C enables us to create, update, read, and delete the files stored on the local file system through our C program. The following operations can be performed on a file.

- Creation of the new file
- Opening an existing file
- Reading from the file
- Writing to the file
- Deleting the file

Functions for file handling

There are many functions in the C library to open, read, write, search and close the file. A list of file functions are given below:

No.	Function	Description
1	fopen()	opens new or existing file
2	fprintf()	write data into the file
3	fscanf()	reads data from the file
4	fputc()	writes a character into the file
5	fgetc()	reads a character from file
6	fclose()	closes the file
7	fseek()	sets the file pointer to given position
8	fputw()	writes an integer to file
9	fgetw()	reads an integer from file
10	ftell()	returns current position
11	rewind()	sets the file pointer to the beginning of the file

Opening File: fopen() and Closing File: fclose()

Program

```
#include<stdio.h>
void main( )
{
FILE *fp ;
char ch ;
fp = fopen("file_handle.c","r") ;  // file open, r is mode
while ( 1 )
{
ch = fgetc ( fp ) ;
if ( ch == EOF )
break ;
printf("%c",ch) ;
}
fclose (fp ) ;  //file close
}
```

We can use one of the following modes in the fopen() function.

Mode	Description
r	opens a text file in read mode
w	opens a text file in write mode
a	opens a text file in append mode
r+	opens a text file in read and write mode
w+	opens a text file in read and write mode
a+	opens a text file in read and write mode
rb	opens a binary file in read mode
wb	opens a binary file in write mode
ab	opens a binary file in append mode
rb+	opens a binary file in read and write mode
wb+	opens a binary file in read and write mode
ab+	opens a binary file in read and write mode

Writing File : fprintf() function

The fprintf() function is used to write set of characters into file. It sends formatted output to a stream.

Program

```
#include <stdio.h>
main(){
  FILE *fp;
  fp = fopen("file.txt", "w");//opening file
  fprintf(fp, "Hello file by fprintf...\n");//writing data into file
  fclose(fp);//closing file
}
```

Reading File : fscanf() function

The fscanf() function is used to read set of characters from file. It reads a word from the file and returns EOF at the end of file.

Program

```
1.  #include <stdio.h>
2.  main(){
3.    FILE *fp;
4.    char buff[255];//creating char array to store data of file
5.    fp = fopen("file.txt", "r");
6.    while(fscanf(fp, "%s", buff)!=EOF){
7.    printf("%s ", buff );
8.    }
9.    fclose(fp);
10. }
```

Output:

```
Hello file by fprintf...
```

Writing File : fputc() function

The fputc() function is used to write a single character into file. It outputs a character to a stream.

Program

```
#include <stdio.h>
main(){
  FILE *fp;
  fp = fopen("file1.txt", "w");//opening file
  fputc('a',fp);//writing single character into file
  fclose(fp);//closing file
}
```

Reading File : fgetc() function

The fgetc() function returns a single character from the file. It gets a character from the stream. It returns EOF at the end of file.

Program

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
char c;
clrscr();
fp=fopen("myfile.txt","r");

while((c=fgetc(fp))!=EOF){
printf("%c",c);
}
fclose(fp);
getch();
}
```

Output:

```
this is simple text message
```

Writing File : fputs() function

The fputs() function writes a line of characters into file. It outputs string to a stream.

Program

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
clrscr();

fp=fopen("myfile2.txt","w");
fputs("hello c programming",fp);

fclose(fp);
getch();
}
```

output

```
hello c programming
```

Reading File : fgets() function

The fgets() function reads a line of characters from file. It gets string from a stream.

Program

```
#include<stdio.h>
#include<conio.h>
void main(){
FILE *fp;
char text[300];
clrscr();

fp=fopen("myfile2.txt","r");
printf("%s",fgets(text,200,fp));

fclose(fp);
getch();
}
```

output

```
hello c programming
```

fseek()

fseek() is used to move file pointer associated with a given file to a specific position.

Program

```
#include <stdio.h>

int main()
{
    FILE *fp;
    fp = fopen("test.txt", "r");

    // Moving pointer to end
    fseek(fp, 0, SEEK_END);

    // Printing position of pointer
    printf("%ld", ftell(fp));

    return 0;
}
```

Output

```
81
```

ftell()

ftell() in C is used to find out the position of file pointer in the file with respect to starting of the file.

Program

```
#include<stdio.h>

int main()
{
    /* Opening file in read mode */
    FILE *fp = fopen("test.txt","r");
      char string[20];
    fscanf(fp,"%s",string);

    /* Printing position of file pointer */
    printf("%ld", ftell(fp));
    return 0;
}
```

Output

```
07
```